

SIMATS ENGINEERING

ASSESSMENT TOOLS

COURSE NAME: Theory of Computation
CSA1304

COURSE CODE:

COURSE OUTCOME COVERED

CO3:Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5
Marks:25
Duration: 1 Hour

Total

CASE STUDY

Code of Conduct:

I R Yuvann Nithish(192411267). (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Yuvann Nithish R

SIMATS ENGINEERING

ASSESSMENT TOOLS

1.Designing a Grammar for Arithmetic Expressions

A compiler development team is creating a parser for a calculator application that supports addition, subtraction, multiplication, division, and parentheses. The initial grammar produces ambiguity for expressions such as $a+b*c$.

Case Study Question:

Develop an optimized context-free grammar (CFG) that correctly enforces operator precedence and associativity for the given arithmetic expressions. Explain how your grammar removes ambiguity and supports efficient parsing.

2. Mini Programming Language Parser Development

startup is designing a small scripting language for automating IoT devices. The language supports variable declarations, conditional statements, and loops. However, the parser frequently reports syntax conflicts during compilation.

Case Study Question:

Construct a CFG for the scripting language constructs below and optimize it to eliminate left recursion and reduce parsing conflicts. Demonstrate how the optimized grammar improves parser efficiency.

3. Grammar Optimization for SQL Query Validation

A database company is developing a query validation tool for simplified SQL commands such as SELECT, FROM, and WHERE. The original grammar contains redundant productions that increase parsing time.

Case Study Question:

Develop a CFG for simplified SQL query syntax and optimize the grammar by removing unnecessary productions and ambiguity. Discuss how the optimized grammar helps in faster syntax analysis.

4. Natural Language Processing Chatbot

A chatbot system processes simple English sentences such as:

Subject + Verb + Object

Example: “Students learn TOC.”

The initial CFG incorrectly accepts grammatically invalid sentences.

Case Study Question:

Design a CFG for the chatbot sentence structure and optimize it to improve language validation accuracy. Explain how the grammar ensures correct sentence formation and rejects invalid inputs.

5. Parser Design for HTML-like Markup Language

A web editor company is building a parser for a simplified markup language with opening and closing tags. The existing grammar fails to validate nested tags correctly.

Case Study Question:

Develop and optimize a CFG for validating properly nested tags in the

markup language. Show how the grammar supports recursive nesting and improves parsing reliability.

SOLUTIONS:

1. Designing a Grammar for Arithmetic Expressions

Problem

The expression:

$$a + b * c$$

is ambiguous in a simple grammar such as:

$$E \rightarrow E + E$$

$$E \rightarrow E - E$$

$$E \rightarrow E * E$$

$$E \rightarrow E / E$$

$$E \rightarrow (E)$$

$$E \rightarrow \text{id}$$

For $a + b * c$, the grammar can produce:

$$(a + b) * c$$

or

$a + (b * c)$

But multiplication must have higher precedence than addition.

Optimized CFG

We divide the grammar into different levels:

$E \rightarrow E + T$

$E \rightarrow E - T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow T / F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{id}$

Where:

$E \rightarrow \text{Expression}$

$T \rightarrow \text{Term}$

$F \rightarrow \text{Factor}$

$\text{id} \rightarrow \text{identifier/number}$

Precedence

The grammar gives:

E

↓

T

↓

F

Therefore:

* and / → higher precedence

+ and - → lower precedence

For:

$a + b * c$

the parser produces:

$a + (b * c)$

rather than:

$(a + b) * c$

Associativity

The productions:

$E \rightarrow E + T$

$E \rightarrow E - T$

make + and - **left associative**.

Similarly:

$T \rightarrow T * F$

$T \rightarrow T / F$

make * and / **left associative**.

For example:

$a - b - c$

is interpreted as:

$(a - b) - c$

Why ambiguity is removed

The original grammar allows multiple parse trees because all operators are at the same level.

The optimized grammar separates operators into precedence levels:

Expression $\rightarrow +, -$

Term $\rightarrow *, /$

Factor $\rightarrow \text{parentheses, identifiers}$

Thus, the parser has only one valid structural interpretation for an expression.

Final Answer

Optimized CFG:

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow (E) \mid \text{id}$

This grammar correctly handles **precedence, associativity, and parentheses**, thereby eliminating ambiguity and making parsing more reliable.

Note: This grammar is unambiguous but still left-recursive.
For an LL(1) parser, it can be further optimized as shown below.

LL(1)-friendly version

$$E \rightarrow T E'$$
$$E' \rightarrow + T E' \mid - T E' \mid \varepsilon$$
$$T \rightarrow F T'$$
$$T' \rightarrow * F T' \mid / F T' \mid \varepsilon$$
$$F \rightarrow (E) \mid \text{id}$$

This version removes **left recursion** and is suitable for efficient **predictive parsing**.

2. Mini Programming Language Parser Development

Suppose our IoT scripting language supports:

- Variable declarations
- Assignment
- Conditional statements
- Loops

For example:

```
int x;
```

```
x = 10;
```

```
if (x > 5) {
    x = x + 1;
}
```

```
while (x < 20) {
    x = x + 1;
}
```

Initial CFG

A possible grammar is:

$$S \rightarrow S ; S$$

$$S \rightarrow \text{int id} ;$$

$$S \rightarrow \text{id} = E ;$$

$$S \rightarrow \text{if} (C) \{ S \}$$

$$S \rightarrow \text{while} (C) \{ S \}$$

$$E \rightarrow E + T$$

$$E \rightarrow T$$
$$T \rightarrow \text{id}$$
$$T \rightarrow \text{number}$$
$$C \rightarrow E > E$$
$$C \rightarrow E < E$$

This grammar contains **left recursion**, especially:

$$S \rightarrow S ; S$$
$$E \rightarrow E + T$$

Left recursion causes problems for recursive-descent and LL parsers.

Optimized CFG

We can remove left recursion.

Program

$$\text{Program} \rightarrow \text{StmtList}$$

Statement List

$$\text{StmtList} \rightarrow \text{Stmt StmtList}'$$
$$\text{StmtList}' \rightarrow \text{Stmt StmtList}' \mid \varepsilon$$

Statements

$$\text{Stmt} \rightarrow \text{Decl}$$
$$\mid \text{Assign}$$
$$\mid \text{IfStmt}$$

| WhileStmt

Declaration

$\text{Decl} \rightarrow \text{int id} ;$

Assignment

$\text{Assign} \rightarrow \text{id} = \text{Expr} ;$

If Statement

$\text{IfStmt} \rightarrow \text{if} (\text{Cond}) \{ \text{StmtList} \}$

While Statement

$\text{WhileStmt} \rightarrow \text{while} (\text{Cond}) \{ \text{StmtList} \}$

Expression

$\text{Expr} \rightarrow \text{Term Expr}'$

$\text{Expr}' \rightarrow + \text{Term Expr}' \mid - \text{Term Expr}' \mid \varepsilon$

$\text{Term} \rightarrow \text{id} \mid \text{number}$

Condition

$\text{Cond} \rightarrow \text{Expr RelOp Expr}$

$\text{RelOp} \rightarrow > \mid < \mid == \mid !=$

Complete optimized CFG

$\text{Program} \rightarrow \text{StmtList}$

$\text{StmtList} \rightarrow \text{Stmt StmtList}'$

$\text{StmtList}' \rightarrow \text{Stmt StmtList}' \mid \varepsilon$

$\text{Stmt} \rightarrow \text{Decl}$

$\mid \text{Assign}$

$\mid \text{IfStmt}$

$\mid \text{WhileStmt}$

$\text{Decl} \rightarrow \text{int id} ;$

$\text{Assign} \rightarrow \text{id} = \text{Expr} ;$

$\text{IfStmt} \rightarrow \text{if} (\text{Cond}) \{ \text{StmtList} \}$

$\text{WhileStmt} \rightarrow \text{while} (\text{Cond}) \{ \text{StmtList} \}$

$\text{Expr} \rightarrow \text{Term Expr}'$

$\text{Expr}' \rightarrow + \text{Term Expr}' \mid - \text{Term Expr}' \mid \varepsilon$

$\text{Term} \rightarrow \text{id} \mid \text{number}$

$\text{Cond} \rightarrow \text{Expr RelOp Expr}$

$\text{RelOp} \rightarrow > \mid < \mid == \mid !=$

Why this grammar is better

The original grammar contains:

$$S \rightarrow S ; S$$
$$E \rightarrow E + T$$

These are left-recursive.

The optimized grammar uses:

$$\text{StmtList} \rightarrow \text{Stmt StmtList}'$$
$$\text{StmtList}' \rightarrow \text{Stmt StmtList}' \mid \varepsilon$$

and:

$$\text{Expr} \rightarrow \text{Term Expr}'$$
$$\text{Expr}' \rightarrow + \text{Term Expr}' \mid - \text{Term Expr}' \mid \varepsilon$$

Therefore:

- Left recursion is removed.
- Recursive-descent parsing becomes easier.
- Parsing conflicts are reduced.
- The grammar is more suitable for LL(1) parsing.
- Syntax analysis becomes more efficient.

Example

Input:

int x;

```

x = 10;
while (x < 20) {
    x = x + 1;
}

```

can be parsed as:

Program

```

└─ StmtList
    │
    ├── Decl
    │
    ├── Assign
    │
    └─ WhileStmt
        └─ StmtList
            └─ Assign

```

3. Grammar Optimization for SQL Query Validation

We consider simplified SQL syntax:

SELECT column FROM table WHERE condition;

Example:

SELECT name FROM students WHERE age > 18;

Basic CFG

Query $\rightarrow$ SELECT ColumnList FROM Table WhereClause ;

ColumnList $\rightarrow$ Column

$\text{ColumnList} \rightarrow \text{Column} , \text{ColumnList}$

$\text{Column} \rightarrow \text{id}$

$\text{Table} \rightarrow \text{id}$

$\text{WhereClause} \rightarrow \text{WHERE Condition}$

$\text{WhereClause} \rightarrow \varepsilon$

$\text{Condition} \rightarrow \text{id Op Value}$

$\text{Op} \rightarrow =$

$\text{Op} \rightarrow >$

$\text{Op} \rightarrow <$

$\text{Op} \rightarrow >=$

$\text{Op} \rightarrow <=$

$\text{Value} \rightarrow \text{id}$

$\text{Value} \rightarrow \text{number}$

This grammar can be simplified by combining redundant productions.

Optimized CFG

$\text{Query} \rightarrow \text{SELECT ColumnList FROM id WhereClause ;}$

$\text{ColumnList} \rightarrow \text{id ColumnList}'$

$\text{ColumnList}' \rightarrow , \text{id ColumnList}' \mid \varepsilon$

$\text{WhereClause} \rightarrow \text{WHERE Condition} \mid \varepsilon$

$\text{Condition} \rightarrow \text{id Op Value}$

$\text{Op} \rightarrow = \mid > \mid < \mid >= \mid <=$

$\text{Value} \rightarrow \text{id} \mid \text{number}$

Example 1

SELECT name FROM students;

Derivation:

Query

$\rightarrow \text{SELECT ColumnList FROM id WhereClause} ;$

$\rightarrow \text{SELECT id FROM id } \varepsilon ;$

Example 2

SELECT name, age FROM students WHERE age > 18;

The parser recognizes:

SELECT

name, age

FROM

students

WHERE

age > 18

Optimization performed

Instead of:

ColumnList $\rightarrow$ Column

ColumnList $\rightarrow$ Column , ColumnList

Column $\rightarrow$ id

we use:

ColumnList $\rightarrow$ id ColumnList'

ColumnList' $\rightarrow$, id ColumnList' | ϵ

This reduces unnecessary intermediate non-terminals.

Similarly, instead of:

Op $\rightarrow$ =

Op $\rightarrow$ >

Op $\rightarrow$ <

Op $\rightarrow$ >=

Op $\rightarrow$ <=

we use:

Op $\rightarrow$ = | > | < | >= | <=

Benefits

The optimized grammar:

- Removes redundant productions.
- Reduces the number of grammar rules.
- Makes the parse structure simpler.
- Reduces parsing overhead.
- Makes predictive parsing easier.
- Improves syntax validation speed.

Final CFG

$\text{Query} \rightarrow \text{SELECT ColumnList FROM id WhereClause ;}$

$\text{ColumnList} \rightarrow \text{id ColumnList'}$

$\text{ColumnList'} \rightarrow , \text{id ColumnList'} \mid \epsilon$

$\text{WhereClause} \rightarrow \text{WHERE Condition} \mid \epsilon$

$\text{Condition} \rightarrow \text{id Op Value}$

$\text{Op} \rightarrow = \mid > \mid < \mid >= \mid <=$

$\text{Value} \rightarrow \text{id} \mid \text{number}$

4. Natural Language Processing Chatbot

The chatbot needs to recognize simple English sentences following:

Subject + Verb + Object

Example:

Students learn TOC.

A simple grammar such as:

$$S \rightarrow NP VP$$
$$NP \rightarrow N$$
$$VP \rightarrow V NP$$

may accept incorrect combinations such as:

Students learns TOC

because it doesn't enforce subject-verb agreement.

Optimized CFG

We can separate singular and plural subjects.

$$S \rightarrow \text{PluralS} \mid \text{SingularS}$$
$$\text{PluralS} \rightarrow \text{PluralNP PluralVP}$$
$$\text{SingularS} \rightarrow \text{SingularNP SingularVP}$$

Noun phrases

$$\text{PluralNP} \rightarrow \text{PluralN}$$
$$\text{SingularNP} \rightarrow \text{SingularN}$$

Nouns

PluralN $\rightarrow$ Students | Teachers | Developers

SingularN $\rightarrow$ Student | Teacher | Developer

Verbs

PluralVP $\rightarrow$ PluralV Object

SingularVP $\rightarrow$ SingularV Object

PluralV $\rightarrow$ learn | teach | develop

SingularV $\rightarrow$ learns | teaches | develops

Object

Object $\rightarrow$ TOC | Mathematics | Programming

Complete CFG

S $\rightarrow$ PluralS | SingularS

PluralS $\rightarrow$ PluralNP PluralVP

SingularS $\rightarrow$ SingularNP SingularVP

PluralNP $\rightarrow$ PluralN

SingularNP $\rightarrow$ SingularN

PluralN $\rightarrow$ Students | Teachers | Developers

SingularN $\rightarrow$ Student | Teacher | Developer

PluralVP $\rightarrow$ PluralV Object

SingularVP $\rightarrow$ SingularV Object

PluralV → learn | teach | develop

SingularV → learns | teaches | develops

Object → TOC | Mathematics | Programming

Valid sentence

Students learn TOC.

Derivation:

S

→ PluralS

→ PluralNP PluralVP

→ Students PluralVP

→ Students learn Object

→ Students learn TOC

Another valid sentence

Student learns TOC.

S

→ SingularS

→ SingularNP SingularVP

→ Student SingularVP

→ Student learns Object

→ Student learns TOC

Invalid sentence

Students learns TOC.

The grammar expects:

Plural Subject → Plural Verb

So:

Students → learn

not:

Students → learns

Similarly:

Student learn TOC

is rejected because:

Singular Subject → Singular Verb

requires:

Student learns TOC

Benefits

The optimized grammar:

- Enforces Subject-Verb agreement.
- Reduces invalid sentence acceptance.
- Provides better language validation.
- Separates singular and plural structures.
- Makes parsing more predictable.

Important: A full English grammar is far more complex than this. This CFG is appropriate for the **restricted sentence structure specified in the case study.**

5. Parser Design for HTML-like Markup Language

Suppose the markup language has tags such as:

`<book>content</book>`

and nested tags:

`<book>`

`<title>TOC</title>`

`<author>John</author>`

`</book>`

The important rule is:

Every opening tag must have a corresponding closing tag,
and nested tags must close in the reverse order in which they
were opened.

Basic CFG

We can define:

$S \rightarrow \text{Element}$

Element $\rightarrow$ `<tag>` Content `</tag>`

Content $\rightarrow$ Element Content

Content $\rightarrow$ text

Content $\rightarrow \epsilon$

This grammar is recursive, allowing elements to contain other elements.

However, using a generic tag in both opening and closing positions does not by itself guarantee that the two tag names are identical.

Optimized CFG for a fixed tag set

For a simplified language containing:

book

title

author

we can write:

Document $\rightarrow$ Element

Element $\rightarrow$ Book

| Title

| Author

Book $\rightarrow$ `<book>` Content `</book>`

Title $\rightarrow$ `<title>` Text `</title>`

Author $\rightarrow$ `<author> Text </author>`

Content $\rightarrow$ Element Content

| Text

| ϵ

Text $\rightarrow$ text

Example

Input:

`<book>`

`<title>TOC</title>`

`<author>John</author>`

`</book>`

can be represented as:

Document

|

Element

|

Book

|

Content

/ \

Title Author

The recursive rule:

Content → Element Content

allows:

Book

↓

Title

↓

Author

and even deeper nesting:

<book>

 <chapter>

 <section>

 <title>TOC</title>

 </section>

 </chapter>

</book>

How recursion validates nesting

Consider:

<book>

 <title>TOC</title>

</book>

The grammar generates:

Book

→ <book> Content </book>

→ <book> Title </book>

→ <book> <title> Text </title> </book>

The closing </title> occurs before </book>.

This matches the nesting structure.

Invalid example

<book>

 <title>TOC</book>

</title>

This cannot be derived correctly because the closing tags are in the wrong order.

Correct:

<book>

 <title>TOC</title>

</book>

Incorrect:

<book>

 <title>TOC</book>

</title>

More general CFG

For conceptual understanding, the grammar can be written as:

Document $\rightarrow$ Element

Element $\rightarrow$ $\langle$ Tag $\rangle$ Content $\langle$ /Tag $\rangle$

Content $\rightarrow$ Element Content

| text Content

| ϵ

where:

Tag $\rightarrow$ book | title | author | chapter

This demonstrates **recursive nesting**.

However, there is an important theoretical point: a plain CFG cannot generally enforce that an arbitrary opening tag name exactly matches its closing tag name using the simple production

Element $\rightarrow$ $\langle$ Tag $\rangle$... $\langle$ /Tag $\rangle$

because both occurrences of Tag are independent. For a **restricted fixed set of tags**, separate productions such as:

Book $\rightarrow$ $\langle$ book $\rangle$ Content $\langle$ /book $\rangle$

Title $\rightarrow$ $\langle$ title $\rangle$ Text $\langle$ /title $\rangle$

solve the matching problem cleanly.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand